Doc-AI Enterprise RAG Chatbot — Hackathon Brief

Oct 31, 2026 · @Saurabh lokhande

A chatbot that answers questions from 12 real PDFs (4,584 pages) using only free, open-source models, with a PDF-and-page citation on every answer.

Problem and solution

Challenge. Build a RAG chatbot over at least 10 PDFs of 200+ pages each. It must OCR scanned pages, use free open-source embeddings and a free vector DB, answer in 2–5 s, and cite PDF name and page.

What we built. Doc-AI ingests PDFs (including scans), indexes them two ways (by meaning and by keyword), and answers questions with a local open-source LLM. Every sentence of an answer is tagged with its source; clicking a citation opens the PDF page with the passage highlighted. The UI also shows the ingestion pipeline, the retrieved chunks for each question, and live latency and quality metrics.

Corpus. 11 real public books and standards (244–640 pages each: NIST, NASA, Sutton & Barto, MacKay, Think Python and others) plus one 30-page image-only scan that can only be read by OCR.

How it works

Ingestion runs once per PDF; every question then searches the shared index in about 0.55 s.

1. **Extract.** PyMuPDF reads text with bounding boxes; pages with almost no text, and large images, are OCR'd with RapidOCR.
2. **Clean.** Unicode is normalised, line-break hyphens joined, running headers and page numbers removed, and language detected.
3. **Chunk.** Sentences are packed into 500-token chunks with a 75-token (15%) overlap, keeping file, pages and bounding box. 4,584 pages → 6,604 chunks.
4. **Index.** Chunks are embedded with bge-small and stored in ChromaDB (HNSW); a BM25 keyword index holds the same chunks.
5. **Retrieve.** A question searches both indexes (top 30 each); results are merged by Reciprocal Rank Fusion and the best 12 are reranked by a cross-encoder.
6. **Answer.** The top 3 passages go to qwen2.5:1.5b, which streams a short answer tagged [S1], [S2]…; tags are checked, mapped to file and page, and scored for grounding.

Tech stack

Everything is free and open source and runs on one laptop; no data leaves the machine.

Part	Choice	Why
PDF text	PyMuPDF	fastest parser, gives bounding boxes for page highlights
OCR	RapidOCR (Tesseract if installed)	pip-only install, runs on CPU
Embeddings	BAAI/bge-small-en-v1.5	reads a full 512-token chunk; built for search
Vector DB	ChromaDB, HNSW (M 32, ef 200/100)	free, local, near-exact at this size
Keyword search	BM25 on a sparse matrix	catches exact terms like "IR-4" that vectors miss
Reranker	ms-marco-MiniLM-L-6-v2 cross-encoder	reads question and passage together for precision
LLM	qwen2.5:1.5b via Ollama	fastest model with correct answers on this CPU
App	FastAPI (streaming, JWT) + React/Vite	integrated into the existing Doc-AI app

Results

On 40 test questions the right page was in the top 3 results 97.5% of the time, and 80% of answers cited the exact gold page. Measured 2026-10-31 on an Intel i5-1135G7 laptop with 8 GB RAM and no GPU.

Measure	Result	Target
Corpus	12 PDFs, 4,584 pages, 6,604 chunks	≥ 10 PDFs × 200 pages
Recall@3 / Recall@5	97.5% / 97.5%	—
MRR@10	0.867	—
Citation accuracy (gold page cited)	80.0%	—

Measure	Result	Target
Citation validity	90.0%	—
Hallucination rate (lexical check)	10.0%	—
Retrieval latency p50 / p95	0.55 s / 0.79 s	—
First answer word p50 / p95	5.0 s / 6.6 s	2–5 s
Full answer p50 / p95	6.7 s / 9.7 s	2–5 s

Hybrid search beat both single methods (dense alone: Recall@3 87.5%). Latency tuning on this CPU cut the median answer from about 31 s to under 7 s, by moving to a smaller model and a shorter prompt. The questions were generated by a separate model from 40 seeded chunks with known pages; the scripts are in the repository.

Demo, repository and setup

Code: github.com/saurabhmj11/Doc-AI-Enterprise-RAG-Chatbot. Full design write-up: `docs/DESIGN_AND_BUILD_GUIDE.md`.

Demo flow (about 3 minutes):

1. **Ingestion Pipeline tab:** 12 PDFs, 4,584 pages, 44 OCR'd pages and 6,604 vectors; drop in a new PDF and watch each stage's progress bar.
2. **Corpus Q&A tab:** ask "What is the difference between TD learning and Monte Carlo methods?" The answer streams with [S1] citations.
3. Click a citation: the PDF page opens with the passage highlighted.
4. The retrieval trace beside the chat shows the latency waterfall and each chunk's dense, BM25 and rerank scores.
5. **Benchmarking & Metrics tab:** the evaluation report and live p50/p95 latency.

Run it (Windows, Python 3.12, Node 18+, Ollama): `ollama pull qwen2.5:1.5b`; in `backend/` create a venv, `pip install -r requirements.txt`, copy `.env.example` to `.env`, then run `scripts/download_corpus.py`, `scripts/make_scanned_sample.py` and `python -m rag.ingest`. Start `start-backend.cmd` and `start-frontend.cmd` and open http://localhost:5173.

Limitations and next steps

- **Full-answer latency is 6.7 s on CPU, above the 5 s target.** A GPU, or setting `GROQ_API_KEY` (hosted open-weights Llama 3.1 8B, about 1 s generation), meets it with no code change.

- **Small local model.** Answers are short (⩽ 45 words) from 3 passages; a faster LLM allows longer, multi-part answers.
- **Evaluation.** 40 synthetic questions and a lexical hallucination check; next is 20–30 human-written questions and an NLI-based check.
- **Reranker.** On this test set the cross-encoder lowered top-1 accuracy (90% → 80%); keeping it is pending a human-written test.
- **Production gaps.** No per-document access control and one ingestion job at a time.

Built with Claude Code as the AI coding assistant; at runtime the chatbot uses only local open-source models.